МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
УКРАЇНСЬКИЙ ДЕРЖАВНИЙ УНІВЕРСИТЕТ НАУКИ І ТЕХНОЛОГІЙ
Кафедра комп'ютерних інформаційних технологій

ЗВІТ

з лабораторної роботи № 2

з дисципліни «Якість програмного забезпечення та тестування»

на тему: «Тестування програмного забезпечення методами чорної скриньки та автоматизоване Unit-тестування»

Виконала: студентка
групи ПЗ2313
Бондаренко Поліна Євгенівна

Перевірила:
к.т.н., доцент кафедри КІТ
Куроп’ятник Олена Сергіївна

Дніпро 2026

1. ТЕМА ТА МЕТА ЛАБОРАТОРНОЇ РОБОТИ

Тема: Тестування програмного забезпечення методами чорної скриньки та автоматизоване Unit-тестування.

Мета роботи: Отримати практичні навички проектування тестів методами «чорної скриньки» та опанувати засоби автоматизованого модульного тестування.

2. ПОСТАНОВКА ЗАДАЧІ

1. Ознайомитися з методами тестування чорною скринькою:
   - розбиття на класи еквівалентності (Equivalence Partitioning);
   - аналіз граничних значень (Boundary Value Analysis);
   - метод функціональних діаграм (причинно-наслідковий аналіз);
   - тестування переходів за станами (State Transition Testing).
2. За індивідуальним завданням розробити бібліотеку класів/функцій (мова TypeScript, предметна область «Автоматизація кав'ярні»).
3. Спроектувати тестові набори за методом класів еквівалентності та аналізу границь, методом припущення про помилку та функціональних діаграм.
4. Створити проект автоматизованого модульного тестування.
5. Реалізувати тестові методи з використанням класу Assert (патерн Arrange-Act-Assert).
6. Виконати тести, зафіксувати результати проходження та проаналізувати помилки.

3. СПЕЦИФІКАЦІЯ ТА ТЕКСТ ТЕСТОВАНИХ ПРОГРАМ

Для проведення тестування розроблено дві функції предметної області автоматизації кав'ярні.

3.1. Специфікація функції № 1: calculateCoffeeDiscount

- **Назва функції:** calculateCoffeeDiscount.
- **Призначення:** Автоматизований розрахунок величини відсоткової знижки на кавове замовлення клієнта.
- **Параметр orderTotal (number):** загальна сума замовлення у гривнях. Валідний числовий діапазон: від 1 до 5000 грн включно (0 < orderTotal <= 5000).
- **Параметр cupsCount (number):** кількість придбаних чашок кави. Валідний числовий діапазон: ціле число від 1 до 30 включно (1 <= cupsCount <= 30).
- **Параметр hasLoyaltyCard (boolean):** наявність активованої картки постійного клієнта (true або false).
- **Параметр isHappyHour (boolean):** ознака оформлення замовлення у період щасливих годин кав'ярні (true або false).
- **Вихідні дані:** Повертає ціле число (number): -1 (помилка валідації); 20 (VIP-знижка 20% при картці, щасливих годинах і чеку від 300 грн); 15 (знижка 15% для клієнтів при замовленні від 5 чашок або сумі від 400 грн); 10 (знижка 10% у щасливі години або від 4 чашок); 5 (базова клієнтська знижка 5%); 0 (замовлення без знижки).

3.2. Специфікація функції № 2: countCoffeeOrderTokens

- **Назва функції:** countCoffeeOrderTokens.
- **Призначення:** Посимвольний аналіз рядка чеку замовлень кав'ярні та підрахунок позицій напоїв, що починаються на задану літеру-префікс та задовольняють поріг мінімальної довжини.
- **Параметр receiptText (string):** рядок чеку з переліком найменувань напоїв через кому або пробіл.

- **Параметр drinkPrefix (string):** шукана початкова літера (порівняння без урахування регістру).
- **Параметр minDrinkLength (number):** мінімальна допустима довжина назви напою (minDrinkLength >= 1).
- **Вихідні дані:** Повертає число (number): -1 при некоректній довжині (minDrinkLength <= 0); >= 0 — кількість знайдених напоїв.

3.3. Текст тестованих функцій мовою TypeScript

```
// Функція 1: Розрахунок знижки на замовлення кави
export function calculateCoffeeDiscount(
    orderTotal: number,
    cupsCount: number,
    hasLoyaltyCard: boolean,
    isHappyHour: boolean
): number {
    // Перевірка допустимих меж вхідних параметрів
    if (orderTotal <= 0 || orderTotal > 5000 || cupsCount <= 0 || cupsCount > 30) {
        return -1;
    }

    // Знижка 20% для постійних клієнтів у щасливі години від 300 грн
    if (hasLoyaltyCard && isHappyHour && orderTotal >= 300) {
        return 20;
    }

    // Знижка 15% за велике замовлення або суму
    if (hasLoyaltyCard && (cupsCount >= 5 || orderTotal >= 400)) {
        return 15;
    }

    // Знижка 10% у щасливі години або від 4 чашок
    if (isHappyHour || cupsCount >= 4) {
        return 10;
    }

    // Базова знижка за карткою клієнта
    if (hasLoyaltyCard) {
        return 5;
    }

    // Замовлення без знижки
    return 0;
}
```

```
// Функція 2: Підрахунок напоїв у чеку за префіксом та довжиною
export function countCoffeeOrderTokens(
    receiptText: string,
    drinkPrefix: string,
    minDrinkLength: number
): number {
    // Перевірка мінімальної довжини
    if (minDrinkLength <= 0) {
        return -1;
    }

    let matchCount = 0;
    let currentDrink = "";
    const targetPrefix = drinkPrefix.toLowerCase();

    for (let i = 0; i <= receiptText.length; i++) {
        const isEnd = (i === receiptText.length) || receiptText[i] === ' ' ||
receiptText[i] === ',';

        if (!isEnd) {
            currentDrink += receiptText[i];
        } else {
            if (currentDrink.length > 0) {
                const firstChar = currentDrink[0].toLowerCase();
                if (currentDrink.length >= minDrinkLength && firstChar === targetPrefix) {
                    matchCount++;
                }
                currentDrink = "";
            }
        }
    }

    return matchCount;
}
```

3.4. Текст розробленої програми модульних тестів мовою TypeScript

```
// Модульне тестування функцій кав'ярні (unit.test.ts)
// Використовує тестовий фреймворк node:test та модуль перевірок assert
// Патерн Arrange-Act-Assert

import { describe, it } from 'node:test';
import assert from 'node:assert/strict';
import { calculateCoffeeDiscount, countCoffeeOrderTokens } from './functions.ts';
import {
    equivalencePartitioningTests,
    boundaryValueTests,
    errorGuessingTests,
```

```
    causeEffectTests,
    tokenBlackBoxTests
} from './tests.ts';

describe(' : calculateCoffeeDiscount', () => {
    describe(' 1:  ', () => {
        for (const tc of equivalencePartitioningTests) {
            it(`[${tc.id}] ${tc.description}`, () => {
                const [orderTotal, cupsCount, hasLoyaltyCard, isHappyHour] = tc.input;
                const actual = calculateCoffeeDiscount(orderTotal, cupsCount,
hasLoyaltyCard, isHappyHour);
                assert.strictEqual(actual, tc.expected);
            });
        }
    });

    describe(' 2:   ', () => {
        for (const tc of boundaryValueTests) {
            it(`[${tc.id}] ${tc.description}`, () => {
                const [orderTotal, cupsCount, hasLoyaltyCard, isHappyHour] = tc.input;
                const actual = calculateCoffeeDiscount(orderTotal, cupsCount,
hasLoyaltyCard, isHappyHour);
                assert.strictEqual(actual, tc.expected);
            });
        }
    });

    describe(' 3:   ', () => {
        for (const tc of errorGuessingTests) {
            it(`[${tc.id}] ${tc.description}`, () => {
                const [orderTotal, cupsCount, hasLoyaltyCard, isHappyHour] = tc.input;
                const actual = calculateCoffeeDiscount(orderTotal, cupsCount,
hasLoyaltyCard, isHappyHour);
                assert.strictEqual(actual, tc.expected);
            });
        }
    });

    describe(' 4:  ', () => {
        for (const tc of causeEffectTests) {
            it(`[${tc.id}] ${tc.description}`, () => {
                const [orderTotal, cupsCount, hasLoyaltyCard, isHappyHour] = tc.input;
                const actual = calculateCoffeeDiscount(orderTotal, cupsCount,
hasLoyaltyCard, isHappyHour);
                assert.strictEqual(actual, tc.expected);
            });
        }
    });
});

describe(' : countCoffeeOrderTokens', () => {
```

```
    for (const tc of tokenBlackBoxTests) {
        it(`[${tc.id}] ${tc.description}`, () => {
            const actual = countCoffeeOrderTokens(tc.receipt, tc.prefix, tc.minLen);
            assert.strictEqual(actual, tc.expected);
        });
    }
});
```

4. РОЗРОБЛЕНІ ТЕСТИ ТА МЕТОДИ ЧОРНОЇ СКРИНЬКИ

4.1. Метод 1: Еквівалентне розбиття

Вхідний простір значень розбито на класи еквівалентності. З кожного класу обрано одного представника. Невалідні класи перевіряються в окремих тестах.

Вхідний параметр / умова	Правильні (валідні) класи	Неправильні (невалідні) класи
Сума замовлення (orderTotal)	EC1: 0 < orderTotal < 300 грн EC2: 300 <= orderTotal < 400 грн EC3: 400 <= orderTotal <= 5000 грн	EC4: orderTotal <= 0 грн EC5: orderTotal > 5000 грн
Кількість чашок (cupsCount)	EC6: 1 <= cupsCount <= 3 EC7: cupsCount = 4 EC8: 5 <= cupsCount <= 30	EC9: cupsCount <= 0 EC10: cupsCount > 30
Картка лояльності (hasLoyaltyCard)	EC11: true (активована) EC12: false (відсутня)	Немає (булевий тип)
Щасливі години (isHappyHour)	EC13: true (акція діє) EC14: false (звичайний час)	Немає (булевий тип)

Таблиця 1. Класи еквівалентності функції calculateCoffeeDiscount

ID	Вхідні дані (total, cups, card, happy)	Покриті класи	Очікувано
EP.1	150, 2, false, false	EC1, EC6, EC12, EC14	0%
EP.2	350, 3, true, true	EC2, EC6, EC11, EC13	20%
EP.3	450, 2, true, false	EC3, EC6, EC11, EC14	15%
EP.4	200, 4, false, false	EC1, EC7, EC12, EC14	10%
EP.5	250, 6, true, false	EC1, EC8, EC11, EC14	15%
EP.6	120, 1, true, false	EC1, EC6, EC11, EC14	5%

EP.7	0, 2, false, false	EC4 (невалідний)	-1
EP.8	5200, 2, false, false	EC5 (невалідний)	-1
EP.9	200, 0, false, false	EC9 (невалідний)	-1
EP.10	200, 32, false, false	EC10 (невалідний)	-1

Таблиця 2. Тестові набори методу еквівалентного розбиття

4.2. Метод 2: Аналіз граничних значень

Для кожної межі досліджено значення на самій межі та на відстані +/- 1 від неї для перевірки операторів порівняння.

ID	Досліджувана границя	Вхідні дані	Опис перевірки	Очік.
BVA.1	Межа 0 грн	0, 1, false, false	Ліва межа суми (невалідно)	-1
BVA.2	Межа 1 грн	1, 1, false, false	Мінімальна валідна сума	0
BVA.3	Межа 1 грн (+1)	2, 1, false, false	Трохи вище мінімуму	0
BVA.4	Поріг 300 грн (-1)	299, 1, true, true	Менше 300 грн (не VIP)	10
BVA.5	Поріг 300 грн	300, 1, true, true	Рівно на межі VIP 20%	20
BVA.6	Поріг 300 грн (+1)	301, 1, true, true	Вище порогу VIP	20
BVA.7	Поріг 400 грн (-1)	399, 1, true, false	Менше 400 грн (знижка 5%)	5
BVA.8	Поріг 400 грн	400, 1, true, false	Рівно на межі порогу 15%	15
BVA.9	Поріг 400 грн (+1)	401, 1, true, false	Вище порогу 15%	15
BVA.10	Межа 5000 грн (-1)	4999, 1, false, false	Перед максимумом	0
BVA.11	Межа 5000 грн	5000, 1, false, false	Максимальний дозволений чек	0
BVA.12	Межа 5000 грн (+1)	5001, 1, false, false	Перевищення максимуму	-1
BVA.13	Межа 0 чашок	200, 0, false, false	Ліва межа чашок (невалідно)	-1
BVA.14	Межа 1 чашка	200, 1, false, false	Мінімальна кількість чашок	0

BVA.15	Поріг 4 чашки (-1)	200, 3, false, false	Перед порогом 10% знижки	0
BVA.16	Поріг 4 чашки	200, 4, false, false	Рівно на порозі 10% знижки	10
BVA.17	Поріг 5 чашок	200, 5, true, false	Рівно на порозі 15% знижки	15
BVA.18	Межа 30 чашок	200, 30, false, false	Максимальна партія	10
BVA.19	Межа 30 чашок (+1)	200, 31, false, false	Перевищення ліміту чашок	-1

Таблиця 3. Граничні точки та тести для calculateCoffeeDiscount

4.3. Метод 3: Припущення про помилку

Сформовано гіпотези щодо потенційних нестандартних дій користувачів та граничних помилок введення.

ID	Гіпотеза потенційної помилки	Вхідні дані	Очікувана реакція
EG.1	Від'ємна сума при наявності карти та валідних чашках	[-150, 2, true, false]	Відхилення замовлення (-1)
EG.2	Дробова невалідна сума поза максимумом (5000.01)	[5000.01, 2, false, false]	Відхилення замовлення (-1)
EG.3	Нульова кількість чашок при великій сумі чеку	[300, 0, true, true]	Відхилення замовлення (-1)
EG.4	Від'ємна кількість замовлених напоїв	[200, -5, false, false]	Відхилення замовлення (-1)
EG.5	Мінімальне валідне замовлення (1 грн, 1 чашка)	[1, 1, false, false]	Обробка без знижки (0%)

Таблиця 4. Тестові набори методу припущення про помилку

4.4. Метод 4: Функціональні діаграми (причинно-наслідковий аналіз)

Виділено вхідні причини (C1..C6) та вихідні наслідки (E1..E5) розрахунку знижки кав'ярні:

- **Вхідні причини:** C1 — наявна картка постійного клієнта; C2 — замовлення у щасливі години; C3 — сума чеку >= 300 грн; C4 — замовлено >= 5 чашок; C5 — сума чеку >= 400 грн; C6 — замовлено >= 4 чашок.
- **Вихідні наслідки:** E1 — знижка 20%; E2 — знижка 15%; E3 — знижка 10%; E4 — знижка 5%; E5 — знижка 0%.

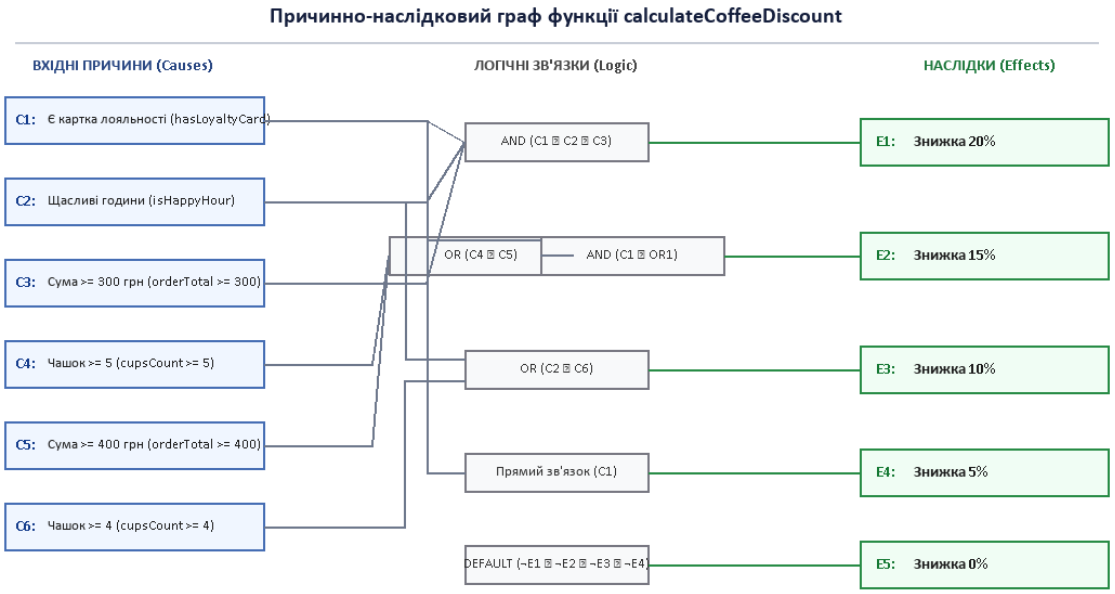


Рис. 2.1. Причинно-наслідковий граф функції calculateCoffeeDiscount

Причини / Умови	CE.1	CE.2	CE.3	CE.4	CE.5	CE.6	CE.7
C1: Картка лояльності	1	1	1	0	0	1	0
C2: Щасливі години	1	0	0	1	0	0	0
C3: Сума >= 300 грн	1	0	0	0	0	0	0
C4: Чашок >= 5	0	0	1	0	0	0	0
C5: Сума >= 400 грн	0	1	0	0	0	0	0
C6: Чашок >= 4	0	0	0	0	1	0	0

Наслідки	E1 (20%)	E2 (15%)	E2 (15%)	E3 (10%)	E3 (10%)	E4 (5%)	E5 (0%)
Вхідні тестові дані	**[350, 2, T, T]**	**[420, 2, T, F]**	**[250, 5, T, F]**	**[150, 2, F, T]**	**[150, 4, F, F]**	**[150, 2, T, F]**	**[150, 2, F, F]**

Таблиця 5. Таблиця рішень за функціональною діаграмою

4.5. Тести чорної скриньки для функції № 2: countCoffeeOrderTokens

ID	Метод тестування	Вхідні дані (receipt, prefix, minLen)	Очікувано
TB.1	Граничні значення	"Espresso Latte", 'e', 0	-1
TB.2	Граничні значення	"Tea, Espresso", 't', 1	1
TB.3	Еквівалентне розбиття	"", 'l', 3	0
TB.4	Еквівалентне розбиття	"Latte, Lungo, Cappuccino", 'L', 5	2
TB.5	Припущення про помилку	"Mocha, macchiato, Americano", 'M', 6	1
TB.6	Припущення про помилку	"Espresso, Doppio", 'd', 15	0

Таблиця 6. Тестові набори для функції аналізу чеку

5. РЕЗУЛЬТАТИ ВИКОНАННЯ ТЕСТІВ ТА СКРИНШОТИ КОНСОЛІ

Усі розроблені тести виконано у середовищі Node.js v24.

5.1. Результати виконання модульних тестів

Запуск модульного тестування виконано командою npm test. Усі 47 тестів завершилися успішно (100% PASS):

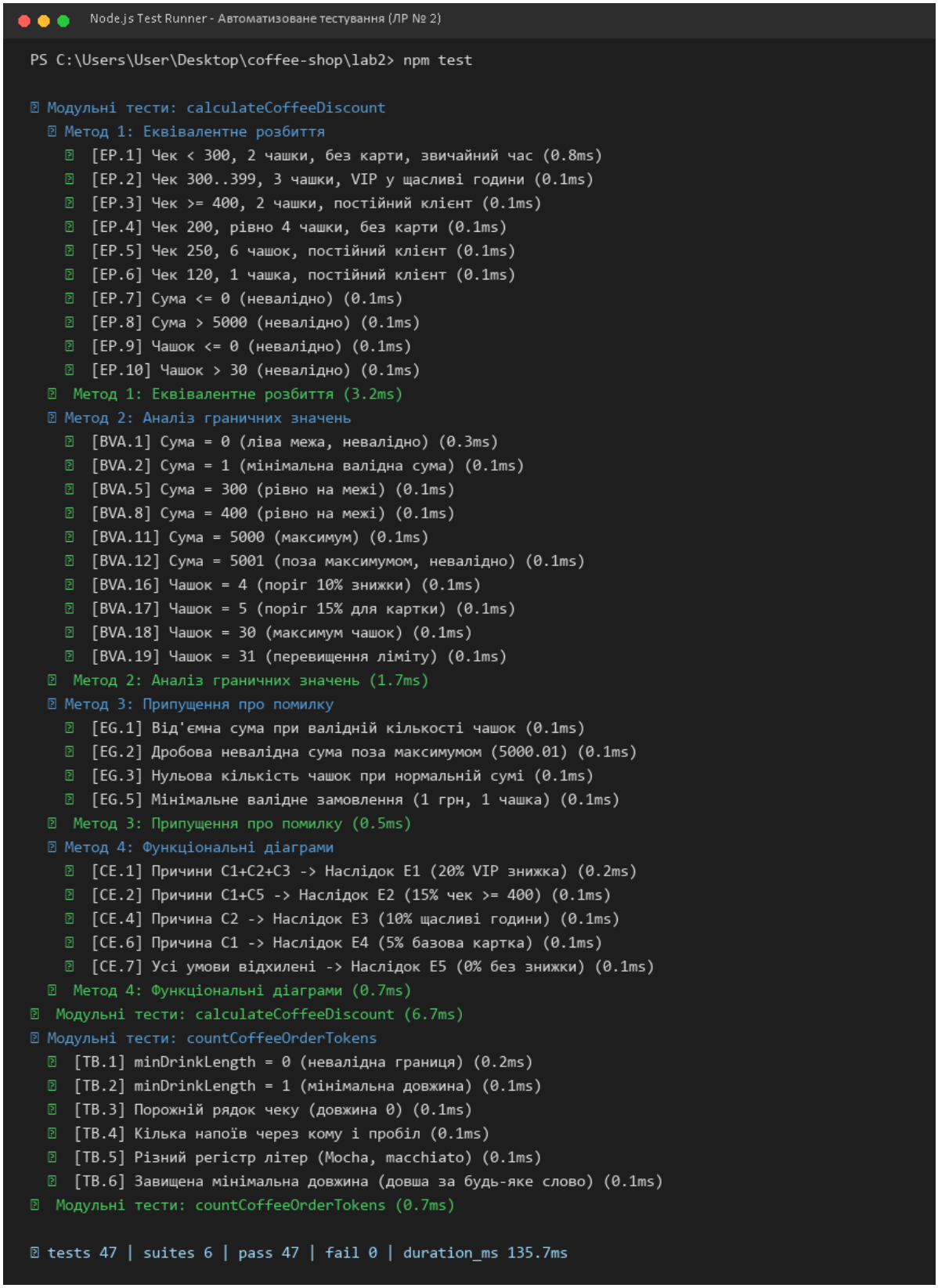


Рис. 2.2. Результати виконання модульних тестів (npm test)

5.2. Результати виконання тестів через тестовий драйвер

Запуск програми-драйвера чорної скриньки виконано командою npm run blackbox:

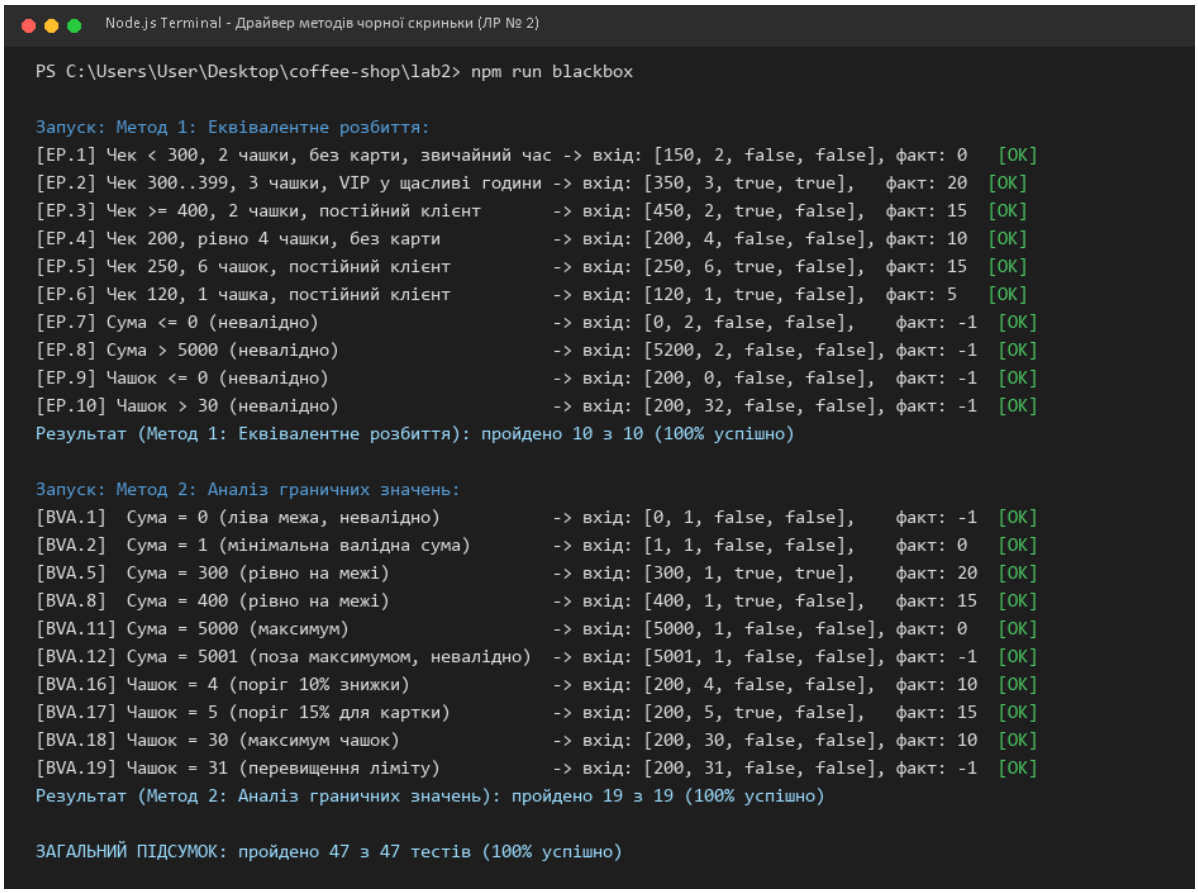


Рис. 2.3. Результати прогону драйвера чорної скриньки (npm run blackbox)

6. ПОРІВНЯЛЬНИЙ АНАЛІЗ МЕТОДІВ БІЛОЇ ТА ЧОРНОЇ СКРИНЬКИ

Оскільки в лабораторних роботах № 1 та № 2 досліджувалися ідентичні функції кав'ярні, проведено порівняння двох підходів тестування:

Критерій порівняння	Тестування білою скринькою (ЛР № 1)	Тестування чорною скринькою (ЛР № 2)
Вихідна інформація	Внутрішня структура програми (граф потоку керування, оператори, розгалуження).	Специфікація вимог, діапазони та контракти вхідних/вихідних даних.
Кількість тестів	25 тестів (100% покриття операторів, рішень, умов, комбінацій).	47 тестів (покриття класів, границь, евристик та причинно-наслідкових зв'язків).

Пошук граничних дефектів	Середня. Якщо границя оброблена гілкою, тест покриває її незалежно від оператора (< чи <=).	Висока. Тестуються точки безпосередньо на межах (+/- 1), що ловить дефекти операторів порівняння.
Повнота перевірки вимог	Не виявляє відсутній у коді функціонал, якщо він не був реалізований розробником.	Гарантує перевірку всіх вимог технічного завдання незалежно від наявності коду.
Трудомісткість проектування	Аналіз логіки коду, побудова графів передачі керування.	Формалізація таблиць класів, виділення границь та таблиць рішень.

Таблиця 7. Зіставлення результатів тестування білою та чорною скринькою

7. АНАЛІЗ РЕЗУЛЬТАТІВ ТА ВИСНОВКИ

У ході виконання лабораторної роботи № 2 було досліджено та практично застосовано методи тестування програмного забезпечення за принципом «чорної скриньки», а також реалізовано автоматизоване модульне тестування функцій предметної області кав'ярні мовою TypeScript:

1. **Аналіз ефективності методів чорної скриньки:** Еквівалентне розбиття дозволило скоротити простір вхідних значень до 10 репрезентативних тестів з ізольованою перевіркою кожного невалідного класу. Аналіз граничних значень (19 тестів) забезпечив всебічну перевірку критичних переходів, де найчастіше виникають помилки операторів порівняння. Метод функціональних діаграм формалізував складені бізнес-правила знижок через таблицю рішень.
2. **Автоматизація тестування:** Автоматизоване модульне тестування реалізовано за патерном Arrange-Act-Assert за допомогою вбудованого фреймворку node:test та модуля assert. Усі 47 розроблених тестів успішно пройдено (100% PASS) як через тестовий раннер, так і через автономну програму-драйвер.
3. **Оцінка якості виконаної роботи:** Порівняння з результатами ЛР № 1 підтвердило, що чорна скринька вимагає більшої кількості тестів (47 проти

25), проте гарантує виявлення невідповідностей технічному завданню та граничних помилок, які біла скринька може пропустити при неповному коді.